Trường Đại học Công nghệ Thông tin Khoa Hệ thống Thông tin	**ĐỀ THI CUỐI KỲ** **HỌC KỲ 2, NĂM HỌC: 2024-2025** **Môn: HỆ QUẢN TRỊ CƠ SỞ DỮ LIỆU** **Thời gian: 90 phút**
(Sinh viên không được phép sử dụng tài liệu - Đề thi gồm có 02 trang)	

Câu 1: [G1.3] (5.0 điểm)

Cho lịch S như sau:

	T1	T2	T3
1	RLock(A)		
2		RLock(C)	
3	WLock(B)		
4		RLock(D)	
5			RLock(B)
6	WLock(C)		
7		WLock(A)	
8			WLock(C)
9	Unlock …	Unlock …	Unlock …

Lưu ý: Dòng Unlock cuối cùng của lịch có nghĩa rằng giao tác nào dùng khóa xong trước sẽ lập tức giải phóng khóa.

a) Dùng đồ thị chờ để đánh giá lịch S có deadlock hay không? **(1.0 điểm)**

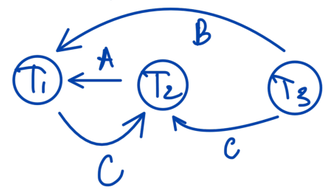
b) Nếu có deadlock, hãy đưa ra 1 giải pháp cụ thể để tránh và 1 giải pháp để giải quyết. **(2.0 điểm)**

c) Thay RLock bằng Read, thay WLock bằng Write và bỏ Unlock của lịch S:

	T1 TS(T1)=10	T2 TS(T2)=20	T3 TS(T3)=30
1	Read(A)		
2		Read(C)	
3	Write(B)		
4		Read(D)	
5			Read(B)
6	Write(C)		
7		Write(A)	
8			Write(C)

Biết các timestamp của các giao tác là: ts(T1) = 10, ts(T2) = 20, ts(T3) = 30. Hãy điều khiển việc truy xuất đồng thời của các giao tác dùng kỹ thuật timestamp từng phần. Nếu có thao tác bị hủy, hãy khởi tạo lại timestamp mới cho đến khi không còn thao tác nào bị hủy nữa. Cho biết lịch khả tuần tự theo thứ tự nào? **(2.0 điểm)**

Ta có đồ thị Wait-for graph:



⇒ Do lịch S có chu trình nên lịch S xảy ra Deadlock.

⇒ Giải pháp giải quyết Deadlock

Chọn T_2 để Rollback, T_2 giải phóng khóa trên DVDL C

- Giải sử T_1 hoặc T_3 xin được khóa trên C
- Giải sử: T_1 xin được dữ liệu C
  - T_1 thực hiện xong → kết thúc → giải phóng dữ liệu trên A, B, C
  - T_3 xin được khóa C
  - → T_3 thực hiện và kết thúc
- T_2 nhận được khóa dữ liệu khi T_1 và T_3 xong

Kết luận: Lịch S khả tuần tự theo thứ tự {T_1, T_3, T_2}.

- Xét đơn vị dữ liệu A:
  Cặp (1,7): $R_1(A)$ .. $W_2(A)$: $T_2 \xrightarrow{A} T_1$
- Xét đơn vị dữ liệu B:
  Cặp (3,5): $W_1(B)$ .. $R_3(B)$: $T_3 \xrightarrow{B} T_1$
- Xét đơn vị dữ liệu C:
  Cặp (2,6): $R_2(C)$ .. $W_1(C)$: $T_1 \xrightarrow{C} T_2$
  Cặp (2,8): $R_2(C)$ .. $W_3(C)$: $T_3 \xrightarrow{C} T_2$

c) Thay RLock bằng Read, thay WLock bằng Write và bỏ Unlock của lịch S:

	T1 TS(T1)=10	T2 TS(T2)=20	T3 TS(T3)=30
1	Read(A)		
2		Read(C)	
3	Write(B)		
4		Read(D)	
5			Read(B)
6	Write(C)		
7		Write(A)	
8			Write(C)

Biết các timestamp của các giao tác là: ts(T1) = 10, ts(T2) = 20, ts(T3) = 30. Hãy điều khiển việc truy xuất đồng thời của các giao tác dùng kỹ thuật timestamp từng phần. Nếu có thao tác bị hủy, hãy khởi tạo lại timestamp mới cho đến khi không còn thao tác nào bị hủy nữa. Cho biết lịch khả tuần tự theo thứ tự nào? **(2.0 điểm)**

* Giải pháp ngăn ngừa deadlock : Wait-die

STT	T_1 $TS(T_1)=10$	T_2 $TS(T_2)=20$	T_3 $TS(T_3)=30$	Ghi chú
1	RL(A)			
2		RL(C)		
3	WL(B)			
4		RL(D)		
5			RL(B)	$TS(T_3) > TS(T_1)$
6	WL(C)		↓	$TS(T_1) < TS(T_2)$
7	↓	WL(A)	die	$TS(T_2) > TS(T_1)$
8	wait	↓		
		die		

- Xét T_2 Rollback, T_1 nhận được khóa trên C
  → T_1 thực hiện xong và kết thúc
- T_2, T_3 thực hiện lại. T_2 nhận được khóa trên A,
  T_3 nhận được khóa trên B.

STT	T_1 $TS(T_1)=10$	T_2 $TS(T_2)=20$	T_3 $TS(T_3)=30$	Ghi chú
1	RL(A)			
2	WL(B)			
3	WL(C)			
4	UL(...)			
5		RL(C)		Dừng lại ở vị trí die lúc trc
6		RL(D)		
7			RL(B)	
8		WL(A)		
9			WL(C)	$TS(T_3) > TS(T_2)$
			↓	
			die	

T_3 thực hiện lại, nhận được khoá trên C sau khi T_2 thực hiện xong và kết thúc.

STT	T_1 $TS(T_1)=10$	T_2 $TS(T_2)=20$	T_3 $TS(T_3)=30$	A $RT(A)=0$ $WT(A)=0$	B $RT(B)=0$ $WT(B)=0$	C $RT(C)=0$ $WT(C)=0$	D $RT(D)=0$ $WT(D)=0$
1	R(A)			$RT(A)=10$ $WT(A)=0$			
2		R(C)				$RT(C)=20$ $WT(C)=0$	
3	W(B)				$RT(B)=0$ $WT(B)=10$		
4		R(D)					$RT(D)=20$ $WT(D)=0$
5			R(B)		$RT(B)=20$ $WT(B)=10$		
6	W(C) ↓						
7	Abort	W(A)					
8			W(C)				

6) $TS(T_1)=10 < RT(C)=20$. Hủy T_1 và khởi tạo lại $TS(T_1)=40$.

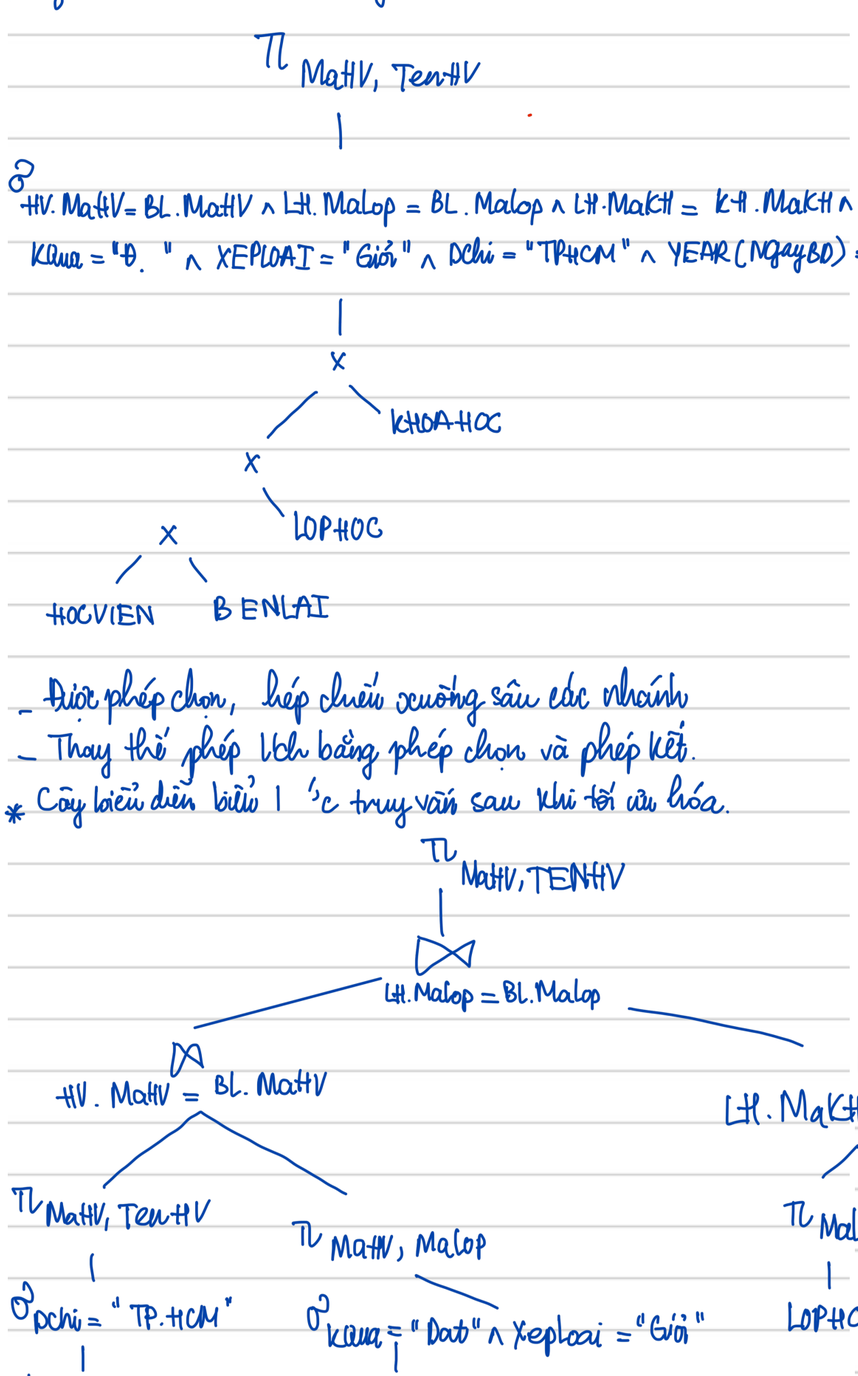
STT	T_1 $TS(T_1)=10$	T_2 $TS(T_2)=20$	T_3 $TS(T_3)=30$	A $RT(A)=0$ $WT(A)=0$	B $RT(B)=0$ $WT(B)=0$	C $RT(C)=0$ $WT(C)=0$	D $RT(D)=0$ $WT(D)=0$
1		R(C)				$RT(C)=20$ $WT(C)=0$	
2		R(D)					$RT(D)=20$ $WT(D)=0$
3			R(B)		$RT(B)=0$ $WT(B)=10$		
4		W(A)		$RT(A)=0$ $WT(A)=40$			
5			W(C)			$RT(C)=20$ $WT(C)=30$	
6	R(A)			$RT(A)=40$ $WT(A)=40$			
7	W(B)				$RT(B)=0$ $WT(B)=40$		
8	W(C)					$RT(C)=20$ $WT(C)=40$	

Lịch khả tuần tự theo thứ tự T_2 T_3 T_1

1. $TS(T_2) > WT(C) \rightarrow T_2$ đọc được C
2. $TS(T_2) > WT(D) \rightarrow T_2$ đọc được D
3. $TS(T_3) > WT(B) \rightarrow T_3$ đọc được B

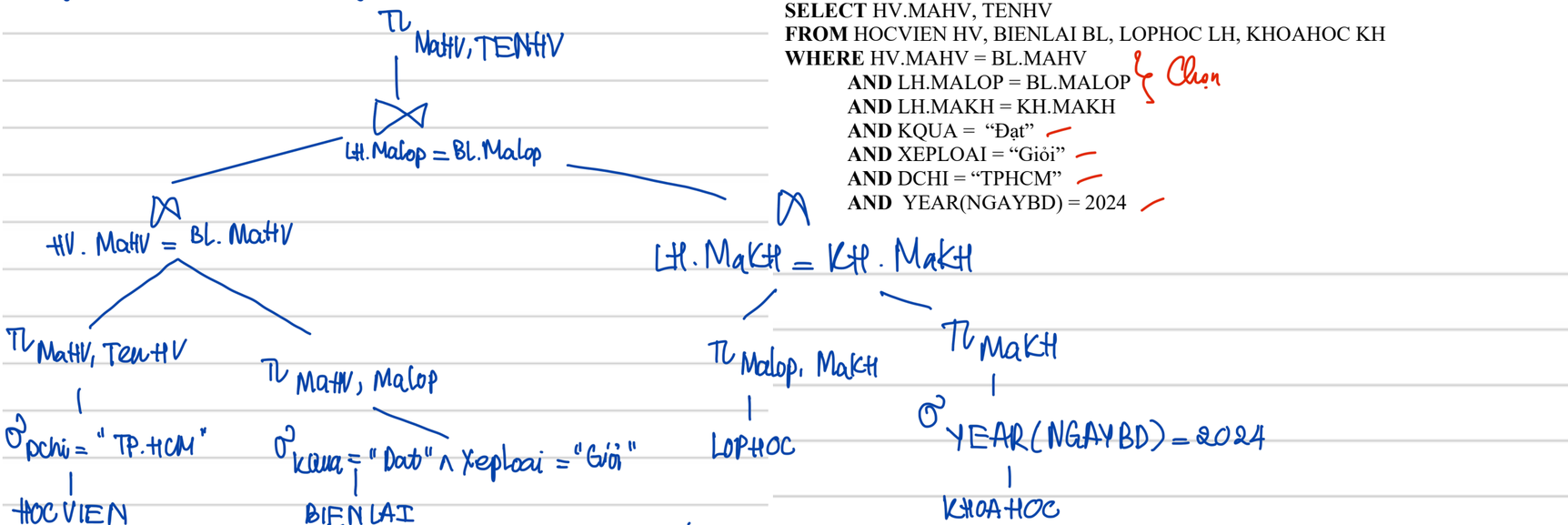
Câu 3:

* Cây biểu diễn biểu thức truy vấn ban đầu.



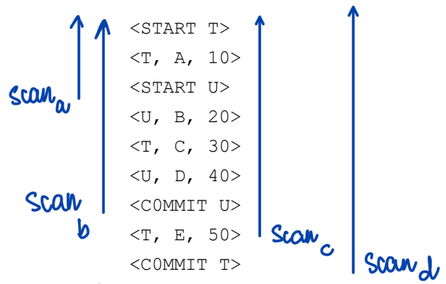
- Được phép chọn, hép chiếu xuống sâu các nhánh
- Thay thế phép tích bằng phép chọn và phép kết.

* Cây biểu diễn biểu thức truy vấn sau khi tối ưu hóa.



Câu 2: **[G1.4] (2.0 điểm)**

Cho tập tin nhật ký sau:



Tiến trình khôi phục của DBMS như thế nào khi dùng phương pháp Undo logging nếu sự cố hệ thống xảy ra ngay sau bước:

Khôi phục từ dưới lên.

a) <START U> **(0.5 điểm)**
b) <COMMIT U> **(0.5 điểm)**
c) <T, E, 50> **(0.5 điểm)**
d) <COMMIT T> **(0.5 điểm)** → Tất cả transaction đều đã COMMIT nên ko có thao tác nào Undo.

Câu 3: **[G1.5, G2.5] (3.0 điểm)**

Một trung tâm muốn tin học hóa công tác quản lý và kết quả học tập của học viên, cũng như theo dõi các lớp học, đã thiết kế lược đồ cơ sở dữ liệu như sau:

KHOAHOC (MAKH, TENKH, NGAYBD, NGAYKT)

HOCVIEN (MAHV, TENHV, NTNS, DCHI, NNGHIEP)

GIAOVIEN (MAGV, TENGV, NTNS, DC)

LOPHOC (MALOP, TENLOP, MAKH, MAGV, SISODK, LTRG, PHHOC)

BIENLAI (SOBL, MALOP, MAHV, DIEM, KQUA, XEPLOAI, TIENNOP)

Hãy mô tả tiến trình tối ưu hóa câu truy vấn sau:

SELECT HV.MAHV, TENHV
FROM HOCVIEN HV, BIENLAI BL, LOPHOC LH, KHOAHOC KH
WHERE HV.MAHV = BL.MAHV
AND LH.MALOP = BL.MALOP } Chọn
AND LH.MAKH = KH.MAKH
AND KQUA = "Đạt"
AND XEPLOAI = "Giỏi"
AND DCHI = "TPHCM"
AND YEAR(NGAYBD) = 2024

a) Xét xảy ra sự cố sau bước <START U>
- Ghi lại A = 10 vì giao tác T chưa hoàn tất.
- ~~Ghi <ABORT T> và <ABORT U> vào log~~

Dừng

b) Xét xảy ra sự cố sau bước <COMMIT U>
- Xét transaction U đã commit nên không hoàn tác
- transaction T vẫn chưa hoàn tất khi gặp lỗi.

⇒ Hoàn tác các giao tác của transaction T

<COMMIT U> : giao tác hoàn tất ⇒ bỏ qua

<U, D, 40> : "

<T, C, 30> : Khôi phục lại giá trị cũ của C

<U, B, 20> : "

<START U> : "

<T, A, 10> : Khôi phục lại giá trị cũ của A

- ~~Gặp <START T> ghi <ABORT T> vào log~~
- Dừng

c) Xét xảy ra sự cố sau bước <T, E, 50>
- Transaction T chưa hoàn tất
- <T, E, 50> : Khôi phục lại giá trị cũ của E

E = 50.

- Dừng.

```
SELECT  HV.MaHV, TENHV
FROM    HOCVIEN HV
WHERE   Dchi = "TP.HCM"
JOIN ( SELECT  MaHV, Malop, MaKH
       FROM  LOPHOC
       JOIN ( SELECT  Malop  FROM BIENLAI WHERE  KQua = "Dat") BL
           ON BL.Malop = LOPHOC.Malop
       JOIN ( SELECT  MaKH FROM KHOAHOC  WHERE  YEAR(NGAYBD) = 2024) KH
           ON KH.MaKH = LOPHOC.MaKH
       WHERE  XEPLOAI = "Gioi"
     ) LH ON HV.MaHV = LH.MaHV
```